

内容中台

9-Agent 内容采集 · 分类 · 发布流水线

使用指南

版本 1.0

2026-04-24

技术栈: TypeScript · Next.js · PostgreSQL · Redis · BullMQ · Groq LLM

目录

1. 系统概览
2. 快速启动
3. 端口与服务
4. 9 个 Agent 的职责
5. 数据流与状态机
6. 三道人工关卡
7. Workbench 操作手册
8. 采集源管理
9. 管理员 API
10. 常见故障排查
11. 性能调优参数
12. 常用命令速查
13. 文件结构

1. 系统概览

内容中台是一套 **9-Agent 自动化内容流水线**：从 RSS / HTML 拉取文章 → LLM 分类打标 → 生成标题 → 选配封面 → 合规审查 → 发布上站 → 社媒分发 → T+1 数据分析。

核心能力

- **自动化** 每小时定时 fan-out 所有激活源，items 一路自动流转
- **人工可控** 3 道人工关卡（合规、发布、分发）不会自动放行
- **可观察** Workbench UI 实时显示每个 Agent 的待处理数、成本、队列状态
- **可插拔** LLM 当前用 Groq 免费档（llama-3.1-8b-instant），可换 Anthropic / OpenAI / 任意 OpenAI 兼容 endpoint
- **幂等安全** BullMQ jobId 去重、SHA256 URL 指纹、SimHash 内容去重

系统拓扑

层	组件	作用
前端	Next.js 14 App Router	站点 + Workbench 控制台
后端	Fastify + BullMQ	API + 9 个 worker 进程
存储	PostgreSQL 16	sources / raw_items / items / agent_runs / analytics_daily / distribution_tasks
队列	Redis 7 + BullMQ	9 个命名队列
对象存储	MinIO (S3 协议)	封面多尺寸图
LLM	Groq (免费)	分类 / 标题 / 合规打分 / 封面文案

2. 快速启动

前置要求

- Node.js ≥ 18, pnpm ≥ 8
- Docker + Docker Compose
- Groq API Key (免费，去 `console.groq.com` 申请)

五步启动

```
# 1. 起基础设施 (Postgres + Redis + MinIO)
pnpm infra:up

# 2. 跑数据库迁移
pnpm migrate

# 3. 种入示例采集源 (6 个历史类 RSS)
pnpm seed

# 4. 起 API 和 Web 前端
pnpm dev
# 会同时启动:
#   API:   http://localhost:4000
#   Web:   http://localhost:3000

# 5. 打开浏览器访问 workbench
open http://localhost:3000/workbench
```

成功标志：API 日志出现 `workers started: 9` 和 `Server listening at http://0.0.0.0:4000`。

环境变量 (.env)

```
DATABASE_URL=postgresql://ch:ch@localhost:5433/ch
REDIS_URL=redis://localhost:6381
S3_ENDPOINT=http://localhost:9002
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
S3_BUCKET=ch-media

GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXX
GROQ_MODEL_LARGE=llama-3.1-8b-instant
GROQ_MODEL_MEDIUM=llama-3.1-8b-instant
GROQ_MODEL_SMALL=llama-3.1-8b-instant

AUTO_PIPELINE_INTERVAL_MS=4000
ALERT_QUEUE_THRESHOLD=100
ALERT_LLM_FAIL_RATE=0.15
```

3. 端口与服务

端口	服务	说明
----	----	----

3000	Next.js Web	站点首页 + <code>/workbench</code> + <code>/admin/*</code>
4000	Fastify API	<code>/admin/*</code> 接口 + 9 worker 进程
5433	PostgreSQL	容器端口 <code>ch-postgres-1</code>
6381	Redis	容器端口 <code>ch-redis-1</code>
9002	MinIO API (S3)	对象上传端点
9003	MinIO Console	浏览器管理界面

端口和常见本地服务（Postgres 5432、Redis 6379）错开，避免和其他项目冲突。

4. 9 个 Agent 的职责

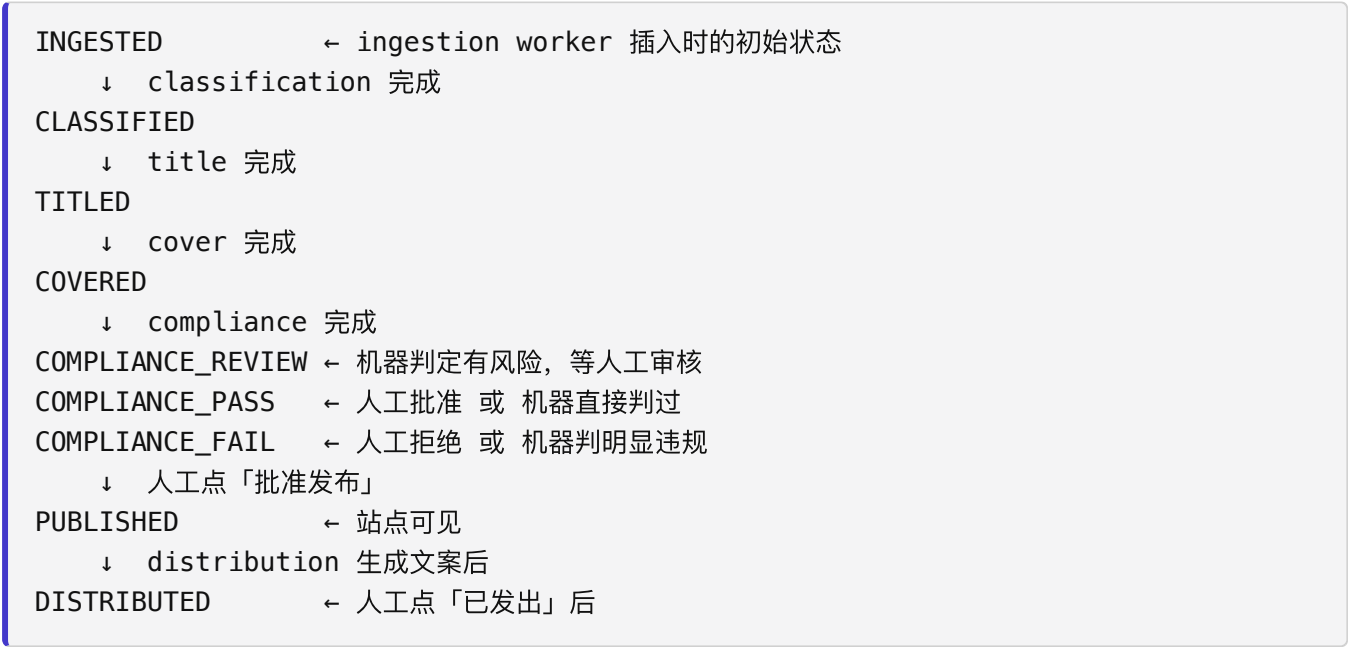
1	2	3	4	5	6	7	8	9
Source 评分	内容采集	分类打标	标题生成	封面选图	合规审查	发布上站	社媒分发	数据分析

#	Agent	干什么	LLM	人工
1	source-scoring	给每个源打质量/稳定性/风险分，决定采集优先级	否	—
2	ingestion	拉 RSS / HTML，去重后存 <code>raw_items</code> + <code>items</code> (<code>INGESTED</code>)	否	—
3	classification	LLM 分类为 16 个预设 category + 抽 tags / keywords	是	—
4	title	LLM 生成 3 候选标题、选最优、写 120~180 字摘要	是	—
5	cover	抽正文首图 → 多尺寸 sharp 裁切 → LLM 生成 alt 文案 → 上传 S3	是	—
6	compliance	黑名单正则 + LLM 对 6 维度（政敏/色/暴/版/医/金）打分	是	⚠ 机器建议复核
7	publishing	写 <code>slug</code> 、触发 ISR revalidate	否	🔒 发布需点批准
8	distribution	为已发布文章生成 X/Twitter 文案	是	🔒 复制文案后手动发

9	analytics	T+1 从 GA4 拉 PV/UV → analytics_daily	否	—
---	-----------	-------------------------------------	---	---

5. 数据流与状态机

items 表状态流转



Worker 交接链路

每个 worker 完成后直接把下一个 job 塞进队列，不等 auto-pipeline 轮询：



auto-pipeline（默认 4 秒轮询）作为兜底，定期检查有没有被漏下的 item，把它们补进队列。

6. 三道人工关卡

△ 合规复核 · 位置：「流水线」tab 合规审查卡片下方

触发条件：LLM 在 6 个维度（政敏/色低俗/暴恐/版权/医疗夸大/金融诱导）里有任意一项评分 ≥2

你要做什么：

- 看风险标签（红色 chip）和文章摘要，判断是否真有问题
- 点 ✓ 批准 → item 转 COMPLIANCE_PASS

- 点 **✖ 拒绝** → 填拒绝原因 → item 转 COMPLIANCE_FAIL , 不会再参与后续流程

 **发布关卡** · 位置：发布上站卡片下方

触发条件：item 到达 COMPLIANCE_PASS 状态

你要做什么：

- 勾选要发布的文章（或全不选即等于批准全部）
- 点 **批准全部 N 篇** / **批准选中 K 篇**
- 后端写 slug、触发 Next.js ISR revalidate, 站点立即可见

 **分发关卡** · 位置：社媒分发卡片下方

触发条件：item 达到 PUBLISHED 后，点「执行」按钮生成文案

你要做什么：

- 点「复制文案」把 LLM 生成的推文复制走
- 手动发到 X / Twitter / 其它平台
- 发完回来点 **✓ 已发出** 记录

7. Workbench 操作手册

地址：`http://localhost:3000/workbench`

7.1 顶部 Header

- **冲刺开始** 日期选择器：修改后所有 Agent 卡片的 "Day N" 标签会以此为基准换算成真实日期
- **总计 / 队列 / 成本** 三个实时数字
-  **紧急停止** 一键把所有 Agent 的 auto 模式关掉、设 `globalStop=true` 。再点一次「▶ 恢复运行」解除

7.2 三个 Tab

流水线

- 9 张卡片从上到下对应 9 个 Agent
- 每张卡片：大数字是待处理数；右边 3 个按钮  自动/手动、 暂停、 执行
-  **自动**：auto-pipeline 会自动把满足条件的 item 塞进这个队列
-  **暂停**：worker 不再处理该队列的新 job
-  **执行**：立刻手动触发一批（当 auto 关闭时用）

采集源

- + 添加来源 展开表单：名称 / Feed URL / platform / ID 四个字段，点紫色「添加」按钮保存
- 每个源右侧 4 按钮：活跃/停用（切换 status）、采集（同步拉取一次）、编辑、删除
- 「删除」如果该源下已有文章，会弹二次确认框问你是否级联删除

队列 & 成本

- 左表：9 个队列的 等待 / 运行 / 完成 / 失败 数
- 右表：最近 24h 每个 Agent 的 成功 / 失败 / 平均耗时 / 累计成本

8. 采集源管理

8.1 支持的 platform

platform	适用	config 必填	备注
rss	RSS / Atom feed	feed_url, limit	默认模式，UI 表单直接支持
html	固定 URL 列表	urls: string[]	不会自动发现文章，得提前列好 URL；UI 表单暂不支持，需 curl 塞

8.2 怎么找一个网站的 RSS URL

1. 猜路径：网址后面挨个试 /feed, /rss, /atom.xml, /index.xml, /?feed=rss2。
WordPress 站基本都有 /feed
2. 源代码：浏览器查看网页 HTML 源码，搜 application/rss+xml，里面的 href 就是
3. RSSHub：rsshub.app 提供 800+ 路由，支持微博/知乎/B 站/Twitter/YouTube 等原本没 RSS 的平台
4. Google News 搜索 RSS：https://news.google.com/rss/search?q=关键词&hl=zh-CN&gl=CN&ceid=CN:zh-Hans
5. 平台套路：
 - Reddit：https://www.reddit.com/r/话题/.rss
 - YouTube 频道：https://www.youtube.com/feeds/videos.xml?channel_id=UCxxx
 - GitHub 用户：https://github.com/用户名.atom

验证技巧：浏览器直接打开那个 URL，应该看到 <?xml 或 <rss 开头的 XML。看到网页、搜索结果页、302 跳转都不是 RSS。

8.3 通过 API 添加 HTML 源

```
curl -X POST http://localhost:4000/admin/sources \
-H "Content-Type: application/json" \
```



```
-d '{
  "platform": "html",
  "external_id": "my-picks",
  "name": "我精选文章",
  "url": "https://example.com",
  "config": {
    "urls": [
      "https://example.com/article/1",
      "https://example.com/article/2"
    ]
  }
}'
```

9. 管理员 API

方法	路径	作用
GET	/admin/sources	列出所有采集源
POST	/admin/sources	新增源
PATCH	/admin/sources/:id	修改源 (name/url/status/config)
DELETE	/admin/sources/:id	删除源 (有 items 时返 409; 加 ? cascade=1 级联删)
POST	/admin/ingest/fanout	立即对所有 active 源 fanout 一轮
POST	/admin/ingest/:id	采集单个源 (同步模式返回 stats; ? async=1 入队)
GET	/admin/stats	总体 dashboard
GET	/admin/items?limit=20	最近入库文章
GET	/admin/pipeline/state	9 Agent 的 auto/paused 状态 + pending 计数
GET	/admin/pipeline/review-queue	待合规审核 items
GET	/admin/pipeline/publish-queue	待发布 items
GET	/admin/pipeline/distribution-queue	待分发任务
POST	/admin/pipeline/approve-review/:itemId	通过合规审核

POST	<code>/admin/pipeline/reject-review/:itemId</code>	拒绝合规审核（需 reason）
POST	<code>/admin/pipeline/approve-publish</code>	批准发布（可传 itemIds 数组选择性发布）
POST	<code>/admin/pipeline/confirm-distribution/:taskId</code>	确认已完成分发
POST	<code>/admin/pipeline/emergency-stop</code>	紧急停止所有自动化
POST	<code>/admin/pipeline/resume</code>	恢复运行
POST	<code>/admin/pipeline/rollback/:itemId</code>	回滚到上一阶段
POST	<code>/admin/pipeline/rerun/:itemId</code>	重跑当前阶段
GET	<code>/admin/pipeline/item-history/:itemId</code>	查看单条 item 的全部 agent_runs 历史
GET	<code>/admin/day7/queues</code>	9 个 BullMQ 队列的详细计数
GET	<code>/admin/day7/agent-runs</code>	近 24h Agent 运行汇总

10. 常见故障排查

10.1 端口 4000 已占用 (EADDRINUSE)

`tsx watch` 偶尔在热重载时旧进程没退出。解决：

```
lsof -ti :4000 | grep -v 5121 | xargs kill -9
pnpm --filter @ch/api dev &
```

5121 是 Chrome Helper 的 PID，它把 4000 加到自己的网络池里但并不监听。

10.2 删除源报 FK 违反约束 (23503)

错误信息：`violates foreign key constraint items_source_id_fkey`。

原因：该源下还有 items。方案：

- **推荐：**点「活跃/停用」改为 inactive，源不会再被采集，文章保留
- **彻底删：**点「删除」→ 第一次 confirm → 后端返 409+itemCount → 第二次 confirm → 加 `?cascade=1` 级联删除 items

10.3 classification 报 "400 Failed to call a function"

Groq 小模型对 function calling 支持差。我们已切换到 JSON mode：把 tool schema 注入 system prompt，让模型直出 JSON 再反序列化。如果仍失败，检查 `anthropic.ts` 里的

useJsonMode 分支。

10.4 LLM 报 429 Rate Limit

Groq 免费档 llama-3.1-8b-instant 只有 6000 TPM。

- LLM worker 并发必须 =1，不能提高
- 每次调用输入不能太长（已经把 MAX_CONTENT 压到 1200~1800 字）
- 想彻底摆脱限流：升级 Groq 付费档，或换 Anthropic / OpenAI

10.5 队列大量 "item not found" 失败

通常是刚刚 TRUNCATE 或 级联删除时，worker 正在处理那些 items。是副作用，不影响新数据。清掉：

```
cd apps/api && ../../node_modules/.bin/tsx scripts/clear-failed.ts
```

10.6 "Feed not recognized as RSS 1 or 2"

填的 URL 不是合法 RSS（比如百度 / 谷歌搜索跳转链接、普通网页）。用 8.2 的方法找到真正的 feed URL，编辑源修正。

10.7 publishing 日志报 revalidate failed 404

Next.js 那边没有 /revalidate 路由。不影响发布：item 已经标 PUBLISHED，只是没触发 ISR 主动刷新。打开页面刷新一次即可。要彻底解决就在 apps/web 加个 /revalidate API Route。

11. 性能调优参数

吞吐量速查

Agent	并发	单条耗时	调优建议
ingestion	6	1-3s	源多就加，纯网络 I/O
classification	1	1-2s	不能加，受 Groq TPM 限制
title	1	2-4s	同上
cover	3	3-6s	主要是图片下载 + sharp，可再加
compliance	1	1-2s	受 Groq TPM 限制
publishing	2	~100ms	DB 写 + ISR，可加高
distribution	3	2-4s	生成文案用 LLM，稍占额度

关键调参文件

参数	位置	推荐值
auto-pipeline 轮询间隔	<code>.env</code> <code>AUTO_PIPELINE_INTERVAL_MS</code>	4000 (4s)
BullMQ 重试 backoff	<code>packages/agents/src/queue.ts</code>	exponential, delay: 500
分类喂给 LLM 的字数	<code>categorize.ts</code> <code>MAX_CONTENT</code>	1200
标题喂给 LLM 的字数	<code>title/generate.ts</code> <code>MAX_CONTENT</code>	1500
合规喂给 LLM 的字数	<code>compliance/score.ts</code> <code>MAX_CONTENT</code>	1800
Ingestion 单源拉取条数	<code>source.config.limit</code>	20

端到端参考

- 6 个 RSS 源，每源 15 条 = 90 条：从点 fanout 到全部发布完，约 2-3 分钟
- 单条新文章：首条出现 ~10 秒；后续每 5-10 秒一条
- Groq 成本：免费档 0 元；换 Anthropic Haiku 约 \$0.0002/条，换 Opus 约 \$0.003/条

12. 常用命令速查

系统

```
pnpm infra:up           # 起 Postgres / Redis / MinIO
pnpm infra:down         # 停
pnpm infra:logs         # 看容器日志
pnpm migrate            # 跑数据库迁移
pnpm seed               # 种入默认 6 个历史 RSS 源
pnpm dev                # 同时启动 API 和 Web
pnpm build              # 全仓库构建
```

数据库（直连）

```
# 连上 psql
docker exec -it ch-postgres-1 psql -U ch ch

# 看 items 状态分布
docker exec ch-postgres-1 psql -U ch ch -c \
  "SELECT status, COUNT(*) FROM items GROUP BY status;"

# 看源列表
docker exec ch-postgres-1 psql -U ch ch -c \
  "SELECT external_id, name, status, last_fetch_at FROM sources;"
```

```
# 清空所有数据（保留 sources）
docker exec ch-postgres-1 psql -U ch ch -c \
  "TRUNCATE items, raw_items, agent_runs, distribution_tasks, analytics_daily
  RESTART IDENTITY CASCADE;"
```

BullMQ 队列

```
# 查看所有队列状态
curl http://localhost:4000/admin/day7/queues | jq

# 清空所有失败任务
cd apps/api && ../../node_modules/.bin/tsx scripts/clear-failed.ts

# 摧毁所有队列（含等待、运行、失败、完成）
cd apps/api && ../../node_modules/.bin/tsx scripts/cleanup-queues.ts

# 重试所有失败任务
cd apps/api && ../../node_modules/.bin/tsx scripts/retry-failed.ts
```

API 调试

```
# 触发一次全局 fanout
curl -X POST http://localhost:4000/admin/ingest/fanout

# 触发单个源同步采集
curl -X POST http://localhost:4000/admin/ingest/<SOURCE_ID>

# 看 pipeline 全局状态
curl http://localhost:4000/admin/pipeline/state | jq

# 紧急停止
curl -X POST http://localhost:4000/admin/pipeline/emergency-stop
```

13. 文件结构

```
ch/
├─ apps/
│   └─ api/                                # Fastify + BullMQ workers
│       └─ src/
│           ├── index.ts                    # API 入口
│           ├── admin.ts                    # /admin/sources, /admin/ingest/*
│           ├── admin-pipeline.ts           # /admin/pipeline/* (UI controls)
│           ├── admin-day3/4/7.ts          # 各阶段专用接口
│           ├── auto-pipeline.ts           # 4s 轮询兜底
│           └─ workers/
│               └─ ingestion/               # RSS/HTML 采集
```

```

├── classification/ # LLM 分类
├── title/ # LLM 标题
├── cover/ # 封面选图 + sharp
├── compliance/ # 合规打分
├── publishing/ # 发布 + ISR
├── distribution/ # 社媒文案
├── source-scoring/ # 源评分
├── analytics/ # GA4 拉取
├── scripts/
│   ├── seed.ts # 种入默认源
│   ├── stress.ts # 压测
│   ├── clear-failed.ts # 清失败任务
│   └── cleanup-queues.ts # 摧毁所有队列
├── web/ # Next.js 14
│   └── src/app/
│       ├── page.tsx # 读者端首页
│       ├── workbench/page.tsx # 工作台 (3 个 tab)
│       ├── admin/ # 每日 admin 页面
│       └── [category]/[slug]/ # 文章详情
├── packages/
│   ├── agents/
│   │   └── src/
│   │       ├── anthropic.ts # LLM 调用封装 (带 JSON mode fallback)
│   │       ├── queue.ts # BullMQ getQueue / startWorker
│   │       └── runner.ts # withRun 包装器 (写 agent_runs)
│   └── db/
│       └── src/client.ts # pg Pool + query()
├── docker-compose.yml # Postgres/Redis/MinIO
├── .env # 配置
├── promote/ # 文档与预览
│   ├── user-guide.html # 本文档
│   ├── workbench-preview.html
│   └── one-week-sprint-plan.md

```